The Codex Workflow: A Developer's Guide to Generating WordPress Themes from a Strategic Framework

Introduction: From Specification to Agentic Generation

The successful application of an AI coding agent, such as the OpenAI Codex Command Line Interface (CLI), necessitates a paradigm shift in the development workflow. The traditional approach of creating a single, monolithic specification document, while essential for human-led projects, must be deconstructed and re-engineered for optimal interaction with an agentic system. This report translates the comprehensive framework for specifying an AI-generated WordPress theme into a practical, step-by-step developer's guide for the Codex CLI.[1] The core thesis is that effective AI-driven development moves away from a "waterfall" model of specification towards an orchestrated series of smaller, context-aware, and verifiable tasks.[2]

In this new paradigm, the developer's role evolves from that of a primary coder to an "AI Director" or "Lead Reviewer." The primary responsibilities become defining tasks with precision, providing the necessary context, rigorously verifying the AI's output, and guiding the subsequent refinement process.[3] This report outlines a complete workflow that mirrors the six-part structure of the foundational framework, demonstrating how each phase of specification is transformed into a distinct stage of the Codex CLI development process. The methodology begins with establishing a persistent strategic context, proceeds through a master plan for code generation, and concludes with a continuous loop of review and refinement.

This process is more than a sequence of prompts; it is an orchestration strategy. It involves a carefully designed flow of information between tasks, using the local file system and a structured command sequence to guide the AI through a logical build process. This transforms the interaction from using a simple tool into directing a junior development

partner, which is the key to leveraging the full potential of agentic coding assistants.

Setting Up the Development Environment

Before initiating the workflow, a foundational development environment must be established. This ensures version control is in place from the outset—a non-negotiable step for managing AI-generated code—and that the Codex CLI is properly configured.

1. **Install the Codex CLI:** The tool can be installed globally using a preferred package manager such as npm or Homebrew.[4]
   - **npm:** npm install -g @openai/codex
   - **Homebrew:** brew install codex
2. **Authenticate:** The first time the codex command is run, it will prompt for authentication. It is recommended to sign in with a ChatGPT account to use Codex as part of a Plus, Pro, Team, or Enterprise plan.[4]
3. **Initialize the Project:** Create a new directory for the WordPress theme and initialize a Git repository within it. All subsequent codex commands should be run from this root directory.
   Bash
   ```
   mkdir my-codex-theme
   cd my-codex-theme
   git init
   ```

Part I: Establishing the Strategic Context for Codex

A primary challenge in any extended, multi-step interaction with an AI model is the potential for context degradation, or "AI amnesia." Without a persistent and easily accessible source of truth, the model may lose track of high-level strategic goals when focused on low-level implementation details. For instance, the core purpose of "Lead Generation" might be forgotten when the task is to style footer links. The solution is to create a persistent, machine-readable strategic brief that the AI can reference throughout the project.

The PROJECT_BRIEF.md File

The most effective practice for maintaining strategic context is to distill the entirety of the project's strategic blueprint into a single, well-structured Markdown file named PROJECT_BRIEF.md.[1] This file serves as the project's persistent "brain" and should be placed in the root of the theme directory. It reformats the human-centric narrative of a business document into a structure optimized for an AI agent, using clear headings, lists, and tables to reduce ambiguity and ensure the strategic constraints are easily parsable.[7] This act of translation is a critical step that prevents misinterpretation of the project's core mission.

The PROJECT_BRIEF.md file should contain the following sections, derived directly from the strategic framework [1]:

- **Primary Purpose:** A clear declaration of the website's main function (e.g., Lead Generation, E-commerce Sales, Brand Authority).
- **Target Audience / User Personas:** Detailed profiles of the end-users, including their demographic and psychographic profiles, technical proficiency, and primary goals on the site.
- **SMART Goals:** A structured table outlining the Specific, Measurable, Achievable, Relevant, and Time-bound goals for all key stakeholders (Business, Users, Content Managers). This table provides concrete, trackable objectives that will guide functional development.

Crafting the Initial Context-Setting Prompt

Once the PROJECT_BRIEF.md file is created, the first interaction with Codex should be to load this context. The entire theme generation process should be treated as a single, continuous session initiated from the project's root directory.

The initial prompt establishes the PROJECT_BRIEF.md as the foundational context for all future tasks:

Bash

```
codex "I am starting a new WordPress theme project. Read the attached PROJECT_BRIEF.md file. This file contains the complete strategic blueprint. Acknowledge that you have understood the primary purpose of the website, the target audience, and the key business goals. We will refer to this file as the 'source of truth' for all subsequent tasks."
```

This command instructs Codex to internalize the project's "why," which will inform every subsequent "what" and "how." For advanced control, specific models like GPT-5-Codex can be configured via the ~/.codex/config.toml file or the /model command within an interactive session.[4]

Part II: Architecting the Theme Structure with Iterative & Multi-Modal Prompts

With the strategic context established, the workflow proceeds to the first major generative phase: architecting the theme's structure. This involves translating the user experience (UX) blueprint into a tangible file and directory scaffold, leveraging both textual and visual inputs to ensure accuracy.[1]

From Sitemap to Scaffolding

The hierarchical sitemap, which outlines all necessary pages and their relationships, serves as the architectural plan.[1] The initial task for Codex is to execute a series of shell commands (

mkdir, touch) to create this structure automatically. This prompt should also instruct the creation of standard WordPress theme directories to ensure a professional and maintainable organization from the start.[8]

Example Prompt:

Bash

```
codex "Based on the sitemap in PROJECT_BRIEF.md, generate and execute the shell commands to create the necessary directories and empty PHP files for the theme. This must include a standard WordPress theme structure with top-level directories for /assets/css, /assets/js, /assets/images, and /template-parts/."
```

Leveraging Wireframes with Multi-Modal Input

A key feature of the Codex CLI is its ability to accept image inputs, which is invaluable for translating visual layouts into code.[4] Describing a complex page layout in text alone is prone to misinterpretation. Providing a wireframe as a visual schematic constrains the AI's output in a productive way, ensuring the generated structure directly matches the UX designer's intent.[1] This multi-modal approach significantly de-risks the layout generation phase by minimizing ambiguity.

The `--image` (or `-i`) flag is used to pass wireframe images (e.g., homepage.png, single-post.png) directly to Codex. The prompt then instructs the agent to generate the HTML and PHP structure based on the visual layout provided.

Example Multi-Modal Prompt:

Bash

```
codex -i assets/wireframes/homepage.png "Generate the HTML and PHP structure for front-page.php based on the attached wireframe. Use semantic HTML5 tags for the functional regions shown (header, main, footer, aside). Populate the content blocks with the placeholder copy specified for the homepage in PROJECT_BRIEF.md. Do not add any CSS styling yet."
```

Generating Content Placeholders and Empty States

To create a theme that is immediately reviewable and robust, prompts should instruct Codex to implement the specified placeholder content strategy and design for "empty states".[1] This involves populating templates with realistic sample text rather than generic "Lorem Ipsum" and creating user-friendly messages for scenarios where no content exists (e.g., an empty search results page or a blog category with no posts).[1]

This phase should be conducted as an iterative loop. The developer should direct Codex to generate one template file at a time, review the output for accuracy and adherence to the wireframe, commit the successful result to Git, and then proceed to the next template. This aligns with the best practice of breaking large tasks into smaller, more manageable, and verifiable steps.[2]

Part III: Implementing the Visual Identity System

This phase translates the theme's visual style guide into a consistent and maintainable CSS architecture. The primary objective is to establish a systematic approach to styling, avoiding hardcoded values and ensuring the theme is both visually appealing and performant.[1]

Creating the Design Token Foundation

Modern, scalable web design relies on design systems, where core visual properties are defined as "tokens" or variables.[11] The first step in this phase is to instruct Codex to generate a foundational

variables.css file. This file will define the entire brand palette (Primary, Secondary, Accent, and Neutral colors) and the full typography system (font families, a modular font size scale, font weights, and line heights) as CSS Custom Properties.[1]

This workflow uses the AI not just to write CSS, but to enforce design system consistency. By creating the token file first and then referencing it in all subsequent styling prompts, an implicit "contract" is established with the AI, reducing the likelihood of inconsistent, one-off styles appearing in the codebase.

Example Prompt:

Bash

```
codex "Create a new file at assets/css/variables.css. Inside this file, define a :root selector. Within this selector, create CSS Custom Properties for the entire color palette and typography system as specified in PROJECT_BRIEF.md. For example, --primary-color: #0A2A4E; and --font-size-base: 16px;."
```

Implementing a Mobile-First Philosophy

The theme must be responsive and optimized for all screen sizes, with a mobile-first approach being the modern standard.[1] Prompts must explicitly instruct Codex to write CSS for the smallest screens by default and then use

min-width media queries to introduce layout changes for larger viewports at the specified breakpoints.

Example Prompt:

Bash

```
codex "Create a new file, assets/css/layout.css. Import variables.css. All base styles in this file must target mobile screens. At the 768px breakpoint, use a min-width media query to change the main content area to a two-column grid. At the 1024px breakpoint, adjust the grid to a three-column layout."
```

Styling Components and Prioritizing Performance

With the foundation in place, individual components can be styled. Each prompt should instruct Codex to use the variables defined in variables.css, reinforcing the design system. Furthermore, prompts should incorporate the critical directive to prioritize performance and usability over purely aesthetic choices where a conflict exists.[1]

Example Prompt:

Bash

```
codex "Create a new file, assets/css/buttons.css. Import variables.css. Style all.btn-primary elements. The background color must use var(--accent-color). The text color must be var(--neutral-dark). Ensure the color combination meets a WCAG 2.1 AA contrast ratio of at least 4.5:1. The styles must be lightweight and performant."
```

Part IV: Generating Core Functionality and Ensuring Technical Excellence

This section transitions from the theme's appearance to its behavior and technical quality. The prompts here focus on generating the core logic in functions.php, implementing customization options, and systematically enforcing standards for performance, SEO, and accessibility.

Scaffolding functions.php

The functions.php file is the logical core of a WordPress theme. Initial prompts should direct Codex to generate the necessary boilerplate, including:

- **Theme Supports:** Adding essential features like title tags, post thumbnails, and HTML5 support using add_theme_support().
- **Menu Registration:** Defining navigation menu locations with register_nav_menus().
- **Asset Enqueuing:** Correctly enqueuing the stylesheets and JavaScript files created in previous steps using wp_enqueue_style() and wp_enqueue_script(), ensuring scripts are deferred where appropriate to prevent render-blocking.[13]

The Verification-Driven Prompting Method

A powerful technique for ensuring high-quality output is to embed technical standards and verification criteria directly into the generation prompts. This "constraint-based prompting" leverages the AI's knowledge of best practices during the act of creation, rather than requiring the developer to manually fix compliance issues after the fact. This approach effectively integrates quality assurance into the development process, turning the AI into an active participant in maintaining standards.

For every functional requirement, the prompt should include a clause that references a specific, measurable standard. The following table synthesizes the key technical standards from the framework and other best practices into a checklist that can be used to formulate these prompts and manually verify the output.[1]

Category	Standard / Metric	Target Value / Rule
Performance	Google PageSpeed Insights (Mobile)	Score of 90+ [14]
	Largest Contentful Paint (LCP)	Under 2.5 seconds [14]
	Asset Optimization	Minify CSS/JS; Serve images in WebP format; Defer offscreen images.[13]
SEO	HTML5 Semantics	Use <header>, <nav>, <main>, <article>, <footer> appropriately.[15]
	Heading Hierarchy	A single <h1> per page, followed by a logical <h2>, <h3> structure.[15]
Accessibility	Conformance Level	WCAG 2.1 Level AA [16]
	Color Contrast	Minimum 4.5:1 for normal text; 3:1 for large text.[16]
	Keyboard Navigation	All functionality must be operable via keyboard, with no keyboard traps.[20]
	Forms & Images	All form inputs must have corresponding <label> elements; all meaningful images must have alt text.[16]

Example Verification-Driven Prompt:

Bash

```
codex "Generate the PHP code to register a 'Contact Form' shortcode in functions.php. The form must include fields for Name,
```

Email, and Message. Crucially, the generated HTML must be compliant with WCAG 2.1 AA standards, ensuring every input has a correctly associated <label> tag and all functionality is keyboard-navigable."

Part V: The Master Generation Plan: A Sequential Command Workflow

This phase represents the culmination of all preceding strategic, UX, and design work, translating the complete theme architecture into a precise, file-by-file generation sequence. This provides the AI with a master blueprint, removing ambiguity and ensuring the final theme is structured correctly according to WordPress conventions.[1]

The Architectural Decision

The first command in this phase must be a definitive declaration of the theme type, as mandated by the framework.[1]

```
Bash
```

codex "Confirming the architectural direction for this project: This will be a Classic WordPress Theme. All templates must be PHP-based. Do not use Full Site Editing or block-based template files."

The Master Template Generation Sequence

The following table provides the master plan for generating the theme's template files. It translates the WordPress Template Hierarchy specification into a sequence of optimized Codex CLI commands.[1] The

codex exec command is used for these non-interactive, file-generation tasks, allowing for a more efficient, script-like workflow.[4] Each command is self-contained and references the

necessary context files (

PROJECT_BRIEF.md, wireframe images) to ensure the output is accurate and aligned with the project goals.

View/Content Type	Required Template File	Optimized Codex CLI Command & Prompt
Homepage	front-page.php	codex exec "Create front-page.php. It must display the content of the 'Front Page' and a section with the three most recent blog posts. Reference PROJECT_BRIEF.md for goals and assets/wireframes/homepage.png for layout." -i assets/wireframes/homepage.png
Blog Index	home.php	codex exec "Create home.php. It must display a paginated, reverse-chronological list of all posts, showing featured image, title, and excerpt. Implement the main WordPress Loop correctly."
Single Blog Post	single.php	codex exec "Create single.php. It must display the full post content, metadata, and comments section. Use semantic tags like <article> and <header>. Ensure all functionality is keyboard-accessible."
Default Static Page	page.php	codex exec "Create the

		default page.php template. It should display the page title and the full page content from the editor."
"About Us" Page	page-about-us.php	codex exec "Create a custom page template named page-about-us.php. It needs a two-column layout as specified in the wireframe. The right column should query and display a list of 'team member' custom post types." -i assets/wireframes/about.png
Category Archive	category.php	codex exec "Create category.php. It must display the category title and description, followed by a paginated list of posts from that category."
Search Results	search.php	codex exec "Create search.php. Display the search query and a list of matching posts. Include a well-designed 'empty state' message if no results are found, as per the spec in PROJECT_BRIEF.md."
404 Error Page	404.php	codex exec "Create 404.php. Display a user-friendly 'Page Not Found' message, a search bar, and a link to the homepage. The tone

		should be helpful, not technical."

Part VI: The Review, Refinement, and Debugging Loop

AI-driven development is an iterative process.[1] The initial theme generated by the master plan should be considered a strong "Version 1.0" that requires thorough testing, review, and refinement. This final phase establishes a continuous feedback loop where the developer partners with Codex to debug issues, refactor code, and polish the final product.

The First Build and Test

After the initial generation sequence is complete, the theme should be installed in a local WordPress development environment. A comprehensive review must be conducted, comparing the generated theme against the PROJECT_BRIEF.md and the Technical Standards Checklist. Any discrepancies, bugs, or areas for improvement should be systematically documented.

Leveraging Codex for Debugging

Instead of manually troubleshooting issues, the developer should leverage Codex as a debugging partner. By providing detailed error messages, logs, or stack traces directly to the CLI, the developer can accelerate the process of identifying root causes and generating solutions.[2]

Example Debugging Prompt:

```bash
```

```
codex "I'm getting a PHP fatal error in functions.php on line 52 when activating the theme. Here is the full error message from the log: [paste error message here]. Please analyze the code in functions.php, identify the root cause of the error, and provide the corrected code block."
```

To maintain a clean context for the AI, it is important to avoid editing files manually within an active Codex session, as this can confuse the agent's understanding of the codebase's current state.[25] The recommended workflow is to allow Codex to make a change, review it, and then commit the successful change to Git before proceeding with the next task.

Prompting for Refactoring and Refinement

Beyond fixing bugs, Codex can be prompted to improve code quality. This includes tasks like increasing efficiency, improving readability, or adhering to programming principles like DRY (Don't Repeat Yourself).

Example Refactoring Prompt:

Bash

```
codex "Review the main WordPress Loop in home.php. This code is repeated in category.php and search.php. Please refactor this code. Create a new template part file named /template-parts/content-excerpt.php and move the repeated loop content into it. Then, update home.php, category.php, and search.php to use the get_template_part() function to include this new file."
```

This report concludes by reinforcing that the most effective use of the Codex CLI is not a linear, one-shot generation process, but a continuous cycle of **Prompt -> Generate -> Review -> Refine**. This iterative loop, directed by a human expert with a clear strategic vision and rigorous technical standards, is the key to transforming a detailed specification into a professional-grade, custom WordPress theme that successfully achieves its objectives.

Works cited

1. A Comprehensive Framework for Specifying an AI-Generated WordPress Theme.pdf
2. Prompting guide - OpenAI Developers, accessed on September 24, 2025, https://developers.openai.com/codex/prompting/
3. Best Practices for Coding with OpenAI Codex | by Ronny Roeller | NEXT AI Engineering, accessed on September 24, 2025, https://medium.com/collaborne-engineering/best-practices-for-coding-with-ope

nai-codex-6907ba6f7596
4. Codex CLI - OpenAI Developers, accessed on September 24, 2025, https://developers.openai.com/codex/cli/
5. openai/codex: Lightweight coding agent that runs in your terminal - GitHub, accessed on September 24, 2025, https://github.com/openai/codex
6. Codex | OpenAI, accessed on September 24, 2025, https://openai.com/codex/
7. Best practices for prompt engineering with the OpenAI API, accessed on September 24, 2025, https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
8. Make sure to use these web design best practices in 2025 | Lummi, accessed on September 24, 2025, https://www.lummi.ai/blog/web-design-best-practices
9. Theme Structure – Theme Handbook - WordPress Developer Resources, accessed on September 24, 2025, https://developer.wordpress.org/themes/core-concepts/theme-structure/
10. OpenAI Codex CLI Tutorial - DataCamp, accessed on September 24, 2025, https://www.datacamp.com/tutorial/open-ai-codex-cli-tutorial
11. 10 Best Practices for Web Design in 2025 - OneNine, accessed on September 24, 2025, https://onenine.com/best-practices-for-web-design/
12. Ultimate Guide To Choosing Colors for Web Design - Flux Academy, accessed on September 24, 2025, https://www.flux-academy.com/blog/ultimate-guide-to-choosing-colors-for-web-design
13. Google PageSpeed Insights: The Truth About Scoring 100/100 - Kinsta, accessed on September 24, 2025, https://kinsta.com/blog/google-pagespeed-insights/
14. What is Google PageSpeed Score and How to Increase It (2025 Guide) - NitroPack, accessed on September 24, 2025, https://nitropack.io/blog/post/improve-pagespeed-insights-score
15. Semantic HTML: Boost Accessibility & SEO with Proven Best Practices - Saffron Edge, accessed on September 24, 2025, https://www.saffronedge.com/blog/semantic-html/
16. WCAG Level AA Checklist: Your Complete Guide to Web Accessibility - accessiBe, accessed on September 24, 2025, https://accessibe.com/blog/knowledgebase/wcag-checklist
17. Google PageSpeed Insights: a Beginner Guide to Balance Scores with Site Functionality, accessed on September 24, 2025, https://wp-rocket.me/blog/google-psi/
18. PageSpeed Insights Rules - Google for Developers, accessed on September 24, 2025, https://developers.google.com/speed/docs/insights/rules
19. What Is Semantic HTML? And How to Use It Correctly - Semrush, accessed on September 24, 2025, https://www.semrush.com/blog/semantic-html5-guide/
20. Web Content Accessibility Guidelines (WCAG) 2.1 - W3C, accessed on September 24, 2025, https://www.w3.org/TR/WCAG21/
21. The Essential WCAG Checklist for Website Accessibility - AudioEye, accessed on September 24, 2025, https://www.audioeye.com/post/wcag-checklist/

22. WebAIM's WCAG 2 Checklist, accessed on September 24, 2025, https://webaim.org/standards/wcag/checklist
23. WCAG Checklist 2.1 AA and 2.2 AA | Accessible.org, accessed on September 24, 2025, https://accessible.org/wcag/
24. WordPress template hierarchy: understanding its structure and how it works - Hostinger, accessed on September 24, 2025, https://www.hostinger.com/tutorials/wordpress-template-hierarchy
25. OpenAI Codex CLI: Lightweight coding agent that runs in your terminal | Hacker News, accessed on September 24, 2025, https://news.ycombinator.com/item?id=43708025